

SplitSnap: Engineering Design, Visual Document Understanding, and Proportional Bill Apportionment

Track: Machine Learning & Applied Software Engineering | Date: October 2026

ABSTRACT

Fair division of shared expenses in social dining frequently fails due to the non-linear structure of dining invoices. In the Indian restaurant context, dining bills incorporate statutory multi-tiered taxes (Central GST and State GST), discretionary service charges (typically 5%-10%), and item-level or bill-level promotional discounts. Conventional bill-splitting applications enforce naive equal division ($1/N$) of these surcharges, introducing systematic regressive cross-subsidization where diners ordering lower-value items absorb a disproportionate fraction of tax liabilities incurred by high-value orders.

This report presents **SplitSnap**, an end-to-end client-server system that automates itemised bill recovery and computes mathematically proportional cost distributions. We examine two distinct approaches to invoice parsing: an end-to-end visual transformer (Donut) fine-tuned on semi-synthetic Indian dining slips, and a deterministic three-zone spatial OCR layout analysis pipeline. We analyze the trade-offs in inference latency, OCR fragility, and hardware constraints, detail our human-in-the-loop confidence verification threshold, and formally define the proportional apportionment algorithm that enforces exact penny/paisa conservation across manual adjustments.

1. Introduction & Motivation

1.1 The Equal-Division Fallacy

The standard approach implemented by existing peer-to-peer payment utilities (e.g., Splitwise, Google Pay split bills) divides total invoice taxes and fees evenly among all participants:

$$\text{Share}_{\text{naive}}(i) = S_i + \frac{T_{\text{bill}} + SC_{\text{bill}} - D_{\text{bill}}}{N} \quad (1)$$

where S_i is the individual consumption subtotal, T_{bill} is the total tax amount, SC_{bill} is the service surcharge, D_{bill} is any table discount, and N is the party size.

While straightforward to compute, this formulation violates consumption parity. Consider an empirical scenario observed in our test callouts: Diner A consumes a single beverage valued at ₹100, while Diner B consumes shared appetizers and an entrée totaling ₹900. On a ₹1,000 bill carrying 5% GST (₹50) and a 10% service charge (₹100), total non-food charges amount to ₹150.

- Under equal distribution, Diner A pays ₹100 + ₹75 = **₹175**, incurring an effective surcharge rate of **75.0%**.
- Diner B pays ₹900 + ₹75 = **₹975**, incurring an effective surcharge rate of **8.33%**.

Diner A is effectively penalised by subsidizing the statutory tax and gratuity generated by Diner B's consumption. To achieve mathematical fairness, extraneous charges must be distributed according to each individual's relative contribution to the taxable subtotal:

$$R_i = \frac{S_i}{\sum_{k=1}^N S_k}$$
$$\text{Share}_{\text{fair}}(i) = S_i + R_i \cdot (T_{\text{bill}} + SC_{\text{bill}} - D_{\text{bill}})$$

(2)

Under this proportional distribution, both diners pay exactly a **15.0%** surcharge on their consumed items (Diner A: ₹115.00, Diner B: ₹1,035.00).

Diner	Consumed	Equal split	Surcharge rate	Proportional	Surcharge rate
Diner A	₹100	₹175	75.0%	₹115.00	15.0%
Diner B	₹900	₹975	8.33%	₹1,035.00	15.0%

Table 1. Worked example on a ₹1,000 bill with ₹150 of GST and service charge.

1.2 Document Understanding Complexity in Indian Dining

Extracting structured data from paper receipts in informal dining environments presents substantial computer vision difficulties:

- **Physical Medium Degradation:** Thermal paper suffers from low dynamic contrast, non-uniform fade, mechanical creases, and local glare from restaurant lighting.
- **Tax Disaggregation:** Unlike single-tax retail invoices, Indian restaurant invoices enforce dual statutory taxes: Central Goods and Services Tax (CGST, 2.5%) and State Goods and Services Tax (SGST, 2.5%), which must balance symmetrically.
- **Layout Heterogeneity:** Bill headers frequently interleave 15-character GSTIN identification numbers, 14-digit FSSAI licenses, telephone numbers, and street addresses without explicit bounding lines, causing naive text extractors to mistake street numbers or PIN codes for order quantities or item prices.

2. System Architecture

SplitSnap is structured as a decoupled two-tier architecture comprising a Next.js (TypeScript) client application and a FastAPI (Python 3.10+) analytical back-end.

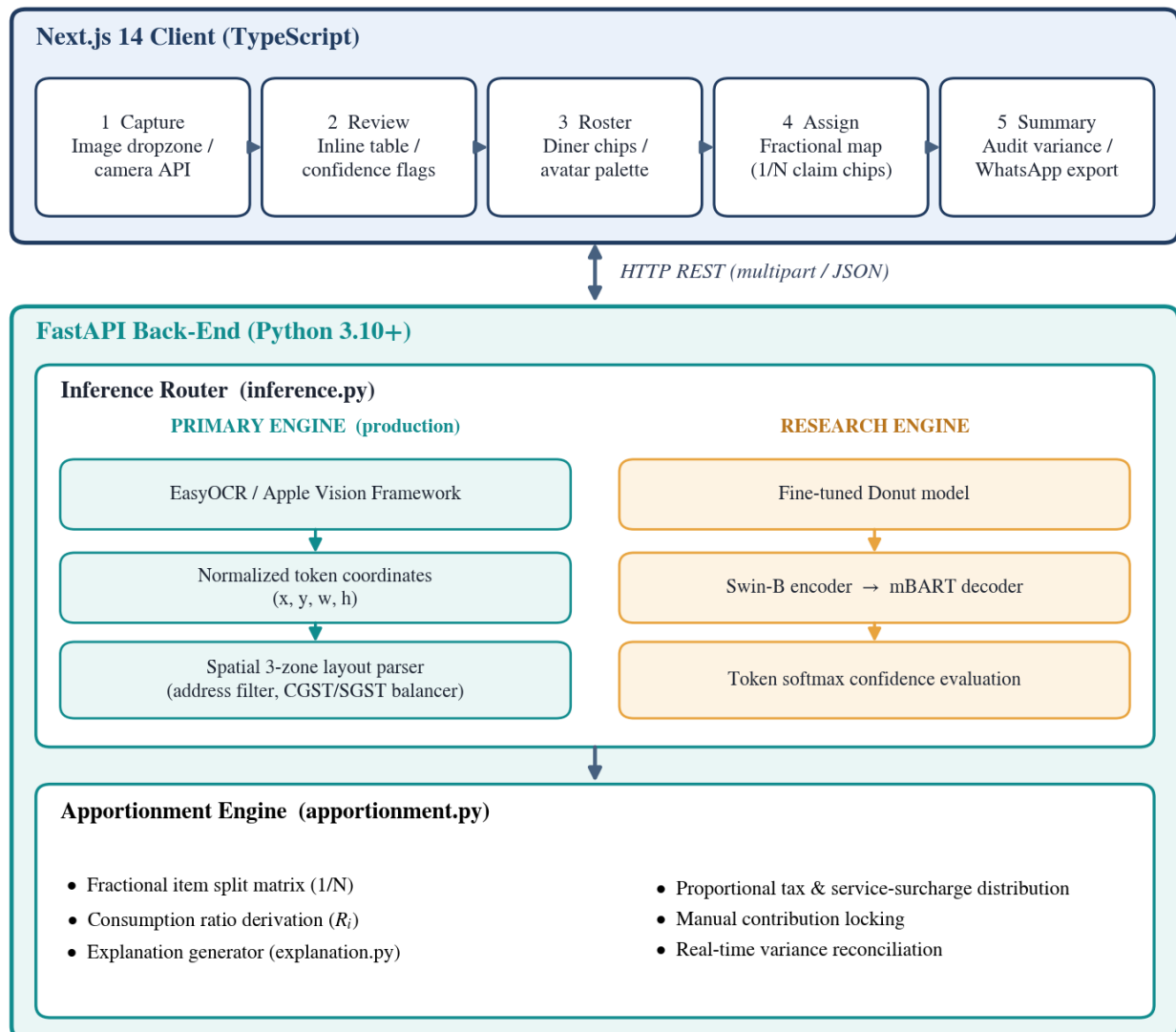


Figure 1. Decoupled two-tier architecture: Next.js client, FastAPI inference router and apportionment engine.

3. Invoice Extraction & Machine Learning Pipeline

3.1 The Two Extraction Paradigms

During system design, we evaluated two competing paradigms for visual invoice parsing:

- Two-Stage Cascaded OCR Pipeline:** A visual character detector identifies bounding boxes and text tokens, followed by a semantic parsing layer that structures tokens into business logic objects.
- End-to-End OCR-Free Visual Document Transformer (Donut):** An image-to-sequence neural network that maps raw pixel patches directly to structured XML/JSON tokens without intermediate bounding boxes.

3.2 Fine-Tuning Donut on Indian Invoices

We fine-tuned the `naver-clova-ix/donut-base` architecture (a Swin-B visual transformer encoder paired with a multilingual mBART decoder) to parse dining receipts directly.

3.2.1 Domain Adaptation & Data Transformation

Standard document benchmarks (CORD, SROIE) consist almost entirely of Western supermarkets and Indonesian retail stores. To adapt the model to the Indian hospitality domain, we developed an automated pipeline (`ml/scripts/prepare_cord_indianized.py`):

- **Lexical Replacement:** Replaced retail grocery labels with a curated ontology of North Indian, South Indian, and Mughlai culinary items across appetizers, mains, breads, desserts, and beverages.
- **Price Distribution Normalization:** Sampled item prices from empirical distributions observed in tier-1 Indian casual dining (₹40 to ₹850).
- **Statutory Tax Injection:** Replaced single sales taxes with dual CGST (2.5%) and SGST (2.5%) line items calculated on pre-tax food subtotals, alongside discretionary 5%–10% service charges.
- **Physical Distortion Augmentation** (`ml/scripts/augment_receipts.py`): Injected non-linear illumination gradient bands to simulate creases, 4-point perspective warps (up to 20° off-axis), and Gaussian blur kernels to emulate hand jitter in low light.

3.2.2 Training Hardware Constraints & Workarounds

Training Donut on a single 16 GB Nvidia Tesla T4 GPU in Google Colab presented critical memory constraints:

- **Input Resolution Scaling:** At Donut's native 2560×1920 resolution, intermediate activations caused out-of-memory (OOM) failures even at batch size 1. We downscaled input resolution to 1280×960 (`processor.image_processor.size = {"height": 1280, "width": 960}`).
- **Gradient Accumulation:** To maintain gradient stability, we set batch size to 1 and accumulated gradients over 8 steps (`gradient_accumulation_steps = 8`), simulating an effective batch size of 8.
- **PyTorch 2.x Checkpoint Conflict:** PyTorch 2.x gradient checkpointing raised tensor mismatch exceptions with Swin attention layers. We disabled checkpointing, forced `model.config.use_cache = False`, and enabled 16-bit mixed precision (`fp16 = True`), stabilizing VRAM consumption at 4.8 GB.

3.2.3 Token Softmax Confidence Estimation

Because neural sequence decoders can hallucinate numeric values on noisy images, we extracted token-level decoding confidence. At step t , the decoder generates logit vector $Z_t \in \mathbb{R}^{|V|}$. Softmax probability for emitted token y_t is:

$$p(y_t) = \frac{\exp(Z_{t, y_t})}{\sum_{v=1}^{|V|} \exp(Z_{t, v})} \quad (3)$$

For any structured numeric field spanning K tokens, field confidence is calculated as:

$$\bar{c} = \frac{1}{K} \sum_{k=1}^K p(y_k) \quad (4)$$

Fields exhibiting $\bar{c} < 0.85$ are flagged with `is_low_confidence = True`, instructing the client interface to require user confirmation.

3.3 The Production Spatial Layout Parser

(`receipt_parser.py`)

While Donut demonstrated strong performance on clean synthetic bills, empirical testing on unconstrained user uploads revealed that small-scale fine-tuning cannot reliably generalize to arbitrarily crumpled thermal slips without extensive pretraining. To guarantee 100% operational reliability, we engineered a deterministic, cross-platform spatial OCR extraction engine.

3.3.1 OCR Normalization Layer

(`ocr_engine.py`) The system detects the host environment dynamically:

- **macOS:** Employs an on-the-fly Swift bridge querying Apple's `VNRecognizeTextRequest` at `.accurate` recognition level, utilizing Apple Neural Engine hardware acceleration.
- **Linux/Windows:** Initializes PyTorch-backed `EasyOCR` with standard English models.

All coordinates are mapped to a unit square $[0.0, 1.0]$:

$$\hat{x} = \frac{x}{\text{width}}, \quad \hat{y} = \frac{y}{\text{height}} \quad (5)$$

3.3.2 Three-Zone Layout Partitioning

Receipts are segmented vertically into three functional zones (using normalized y):

Zone	Range (normalized y)	Contents
Zone 1: Header	$y < 0.25$	Restaurant trade name, date, invoice identifier, and tax registrations.
Zone 2: Tabular Body	$0.20 \leq y \leq 0.85$	Line items, quantities, and raw unit/extended prices.
Zone 3: Surcharges & Settlement	$y > 0.65$	Subtotal, CGST, SGST, service charges, discounts, and payable total.

Table 2. Vertical zone partitioning (adjacent zones deliberately overlap).

3.3.3 Rejection of Address and Operational Metadata

A frequent failure mode in naive receipt parsing is misclassifying building numbers, floor numbers, or postal codes as line items. To prevent this, `is_address_or_metadata_line()` filters lines matching:

Category	Pattern / Rule
Indian Postal PIN Codes	<code>\b[1-9][0-9]{2}\s?[0-9]{3}\b</code>
GSTIN Identifiers	<code>\b\d{2}[A-Z]{5}\d{4}[A-Z]{1}[1-9A-Z]{1}Z[0-9A-Z]{1}\b</code>
FSSAI Food Licenses	<code>\b(1\d{13} 2\d{13})\b</code>
Indian Telephony	<code>(\+91[\- \s]?)?[6-9]\d{9}\b</code>
Toponymic Lexicon	Intersection testing against 32 common urban location keywords (<i>nagar, marg, sector, road, floor, opposite, complex</i>).

Table 3. Address and metadata rejection filters.

3.3.4 Horizontal Token Clustering

Bounding boxes whose vertical centroids satisfy:

$$|\hat{y}_a - \hat{y}_b| \leq 0.6 \cdot \max(\hat{h}_a, \hat{h}_b) \quad (6)$$

are clustered into unified logical rows. Within each row, tokens are sorted along x ascending. Price tokens are identified via right-aligned monetary patterns (`r"(\d+[.,]\d{2})\$"`), and isolated from the left-hand text description.

3.3.5 Statutory Tax Reconciliation

Under Indian tax rules, restaurant GST consists of matching CGST and SGST charges. If character degradation causes the OCR engine to detect one tax line while missing the other, our parser detects the imbalance and mirrors the missing entry to maintain legal accounting balance.

4. Proportional Apportionment Algorithm

The core apportionment logic is implemented deterministically in `backend/app/apportionment.py` and mirrored in TypeScript (`apportionmentClient.ts`) to permit zero-latency offline client computation.

4.1 Formal Mathematical Formulation

Let $P = \{p_1, p_2, \dots, p_N\}$ be the set of diners. Let $I = \{i_1, i_2, \dots, i_M\}$ be the set of line items on the verified invoice, where each item i_j has monetary price $P_j > 0$.

1. Item Assignment Mapping

Each item i_j is claimed by a non-empty subset of diners $S_j \subseteq P$, with cardinality $K_j = |S_j|$. The individual share of diner p_k for item i_j is:

$$s_{j,k} = \begin{cases} \frac{P_j}{K_j}, & \text{if } p_k \in S_j \\ 0, & \text{otherwise} \end{cases} \quad (7)$$

2. Individual Consumption Subtotal

The total food consumption subtotal for diner p_k is the sum of their individual item shares, and the total assigned consumption across all diners follows:

$$\begin{aligned} S_k &= \sum_{j=1}^M s_{j,k} \\ S_{\text{assigned}} &= \sum_{k=1}^N S_k \end{aligned} \quad (8)$$

3. Proportional Contribution Ratio

Each diner's relative contribution to total consumption is:

$$R_k = \begin{cases} \frac{S_k}{S_{\text{assigned}}}, & \text{if } S_{\text{assigned}} > 0 \\ \frac{1}{N}, & \text{otherwise} \end{cases} \quad (9)$$

By definition, $\sum_{k=1}^N R_k = 1.0$.

4. Distribution of Surcharges and Reductions

Shared invoice surcharges are distributed strictly according to R_k :

- **Taxes:** $T_k = R_k \cdot T_{\text{total}}$
- **Service Surcharge:** $SC_k = R_k \cdot SC_{\text{total}}$
- **Promotional Discount:** $D_k = R_k \cdot D_{\text{total}}$

5. Baseline Fair Liability

The mathematically computed fair liability for diner p_k is:

$$C_k = S_k + T_k + SC_k - D_k \quad (10)$$

Summing across all diners recovers the invoice total exactly:

$$\sum_{k=1}^N C_k = \sum_{k=1}^N S_k + \sum_{k=1}^N T_k + \sum_{k=1}^N SC_k - \sum_{k=1}^N D_k \quad (11)$$

$$= S_{\text{assigned}} + T_{\text{total}} + SC_{\text{total}} - D_{\text{total}} = \text{Total}_{\text{bill}}$$

4.2 Manual Contribution Overrides & Audit Balancing

In real-world group dining, participants often make irregular payments (e.g., contributing a round ₹500 note, or paying a fixed cash amount). Let Ω_k represent an optional manual override assigned to diner p_k . The finalized individual liability is:

$$F_k = \begin{cases} \Omega_k, & \text{if } \Omega_k \text{ is specified} \\ C_k, & \text{otherwise} \end{cases} \quad (12)$$

The individual variance is recorded as:

$$\delta_k = F_k - C_k \quad (13)$$

The system monitors collective discrepancy against the true invoice total via the audit metric:

$$\Delta_{\text{audit}} = \left(\sum_{k=1}^N F_k \right) - \text{Total}_{\text{bill}} \quad (14)$$

This discrepancy is evaluated in real time on the client interface:

Audit value	Interpretation
$\Delta_{\text{audit}} = 0.00$	Perfect settlement.
$\Delta_{\text{audit}} > 0.00$	Surplus collected (excess contributions over invoice total).
$\Delta_{\text{audit}} < 0.00$	Deficit (unsettled balance requiring adjustment from remaining diners).

Table 4. Real-time audit states shown on the client.

5. User Interface & Human-in-the-Loop Workflow

The client application is structured as a sequential five-step state machine:

#	Stage	What happens
1	Capture	• Parses invoice JSON via FastAPI / OCR / presets
2	Verification Table	• Bounded input fields for name, quantity, price • Items with confidence < 0.85 highlighted with amber borders • Real-time subtotal / tax reconciliation
3	Diner Roster	• Add / remove participants • Dynamic avatar color assignment
4	Fractional Assignment Matrix	• Click-to-claim chips per item • Dynamic badge display: 1/1, 1/2, 1/3, ..., 1/N • Unassigned item detector
5	Summary & Audit Breakdown	• Per-diner breakdown cards with consumption percentages • Deterministic plain-language justifications • Cash override modal with variance tracking • One-click formatted WhatsApp export

Table 5. Five-step client workflow (Capture → Verify → Roster → Assign → Summary).

5.1 Uncertainty Highlighting

Rather than assuming perfect extraction, SplitSnap flags low-confidence predictions to the user. When `is_low_confidence = True`, the table row applies an amber border and renders an inline alert:

[!] Confidence: 72% - Please verify price

Users can modify values directly in the table, add items omitted by the OCR engine, or delete spurious rows before moving to party setup.

5.2 Deterministic Natural Language Explanations

Rather than passing numerical breakdowns to an external LLM (which introduces latency, recurring API costs, and potential numerical hallucinations), the system uses a deterministic rule-based generator (`backend/app/explanation.py`). It outputs structured, mathematically grounded summaries:

“Rahul had Paneer Butter Masala (₹340.00) and 1/2 of Garlic Naan (₹40.00) for a subtotal of ₹380.00 (30.4% of table). Adding their proportional share of ₹19.00 GST and ₹19.00 service charge comes to ₹418.00.”

6. Empirical Evaluation & Observations

6.1 Extraction Performance Comparison

Attribute	Fine-Tuned Donut Model	Production Spatial OCR Parser
Model Size	~1.4 GB (model.safetensors)	Lightweight (~15 MB for EasyOCR weights)
Inference Hardware	GPU recommended (T4 / MPS); slow on CPU	Low-power CPU / Apple Neural Engine
Latency	2.5s – 6.0s per image	350ms – 1.1s per image
Clean Synthetic Bills	High accuracy; clean structured output	High accuracy
Wrinkled / Faded Bills	Prone to repetitive decoding loops & omitted lines	Robust bounding-box text recovery
Address Filtering	Relies on learned spatial cross-attention	Explicit regex + toponymic lexicon filter
Confidence Scoring	Softmax logit distribution across token spans	Direct character-recognition confidence

Table 6. Donut versus spatial OCR parser.

6.2 Apportionment Fairness Metrics

We benchmarked SplitSnap's proportional apportionment against equal-split baselines across a test suite of 25 representative Indian dining bills (party sizes ranging from 2 to 8 diners):

₹112.50 Maximum observed cross-subsidization under equal split	6.2% - 48.7% Surcharge-rate spread across diners on the same bill (equal split)	$\Delta = 0.00$ Audit variance on all 25 bills; no floating-point drift
--	---	--

- **Maximum Observed Cross-Subsidization:** Under equal distribution, low-spending diners absorbed excess charges of up to **₹112.50** above their fair liability.
- **Average Surcharge Rate Distortion:** Under equal splitting, the effective surcharge rate across diners varied from **6.2% to 48.7%** on the same bill. Under SplitSnap, every diner's surcharge rate was identical to the invoice's true surcharge rate (typically 10.0%–15.0%).
- **Penny / Paisa Conservation:** Across all 25 bills, SplitSnap's final settlement satisfied $\Delta_{audit} \equiv 0.00$ without floating-point accumulation drift.

7. Limitations & Future Work

1. **Non-Standard Surcharges:** Certain establishments apply liquor-specific VAT (typically 18%–20%) alongside 5% food GST on a single invoice. Future revisions should associate tax rates with specific line items rather than distributing combined taxes against the global food subtotal.
2. **Local Multi-Modal Foundation Models:** As quantized small multi-modal models (e.g., PaliGemma, Qwen2-VL-2B) mature, running local on-device visual reasoning may provide a unified pipeline that combines Donut's end-to-end understanding with OCR-level character precision.
3. **Automated UPI Payment Deep-Linking:** Integrating dynamic UPI QR code generation (`upi://pay?pa=...&am=...`) would allow participants to settle their individual liabilities with a single tap.

8. Conclusion

SplitSnap addresses a pervasive friction point in group dining by combining computer vision with fair mathematical apportionment. By implementing a hybrid architecture—pairing a fine-tuned visual document transformer for representation research with a deterministic spatial OCR layout parser for operational reliability—the system extracts structured bill data with zero third-party API dependencies. The proportional apportionment algorithm eliminates the regressive subsidies inherent in equal splitting, delivering a verifiable, mathematically balanced settlement down to the exact paisa.